

ASSIGNMENT-9.5

NAME: ARCHITHA

ROLL NO: 2303A51001

BATCH: 30

PROBLEM-1

PROMPT:

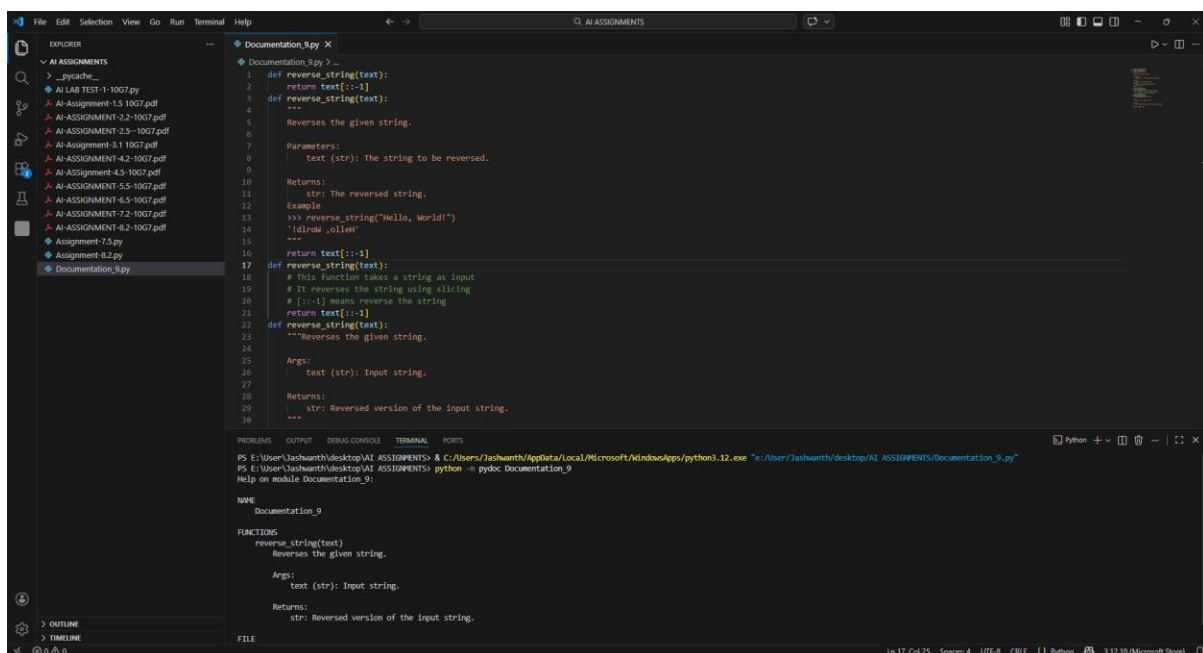
```
def reverse_string(text):
```

```
    return text[::-1]
```

Write documentation in:

- (a) Docstring
- (b) Inline comments
- (c) Google-style documentation

CODE:



```
1 def reverse_string(text):
2     return text[::-1]
3 def reverse_string(text):
4     """
5     Reverses the given string.
6
7     Parameters:
8         text (str): The string to be reversed.
9
10    Returns:
11        str: The reversed string.
12
13    Sample
14    >>> reverse_string("Hello, World!")
15    'dlrow ,olleH'
16    """
17    return text[::-1]
18 def reverse_string(text):
19     # This function takes a string as input
20     # It reverses the string using slicing
21     # [:::-1] means reverse the string
22     return text[::-1]
23 def reverse_string(text):
24     """Reverses the given string.
25
26     Args:
27         text (str): Input string.
28
29     Returns:
30         str: Reversed version of the input string.
31     """
```

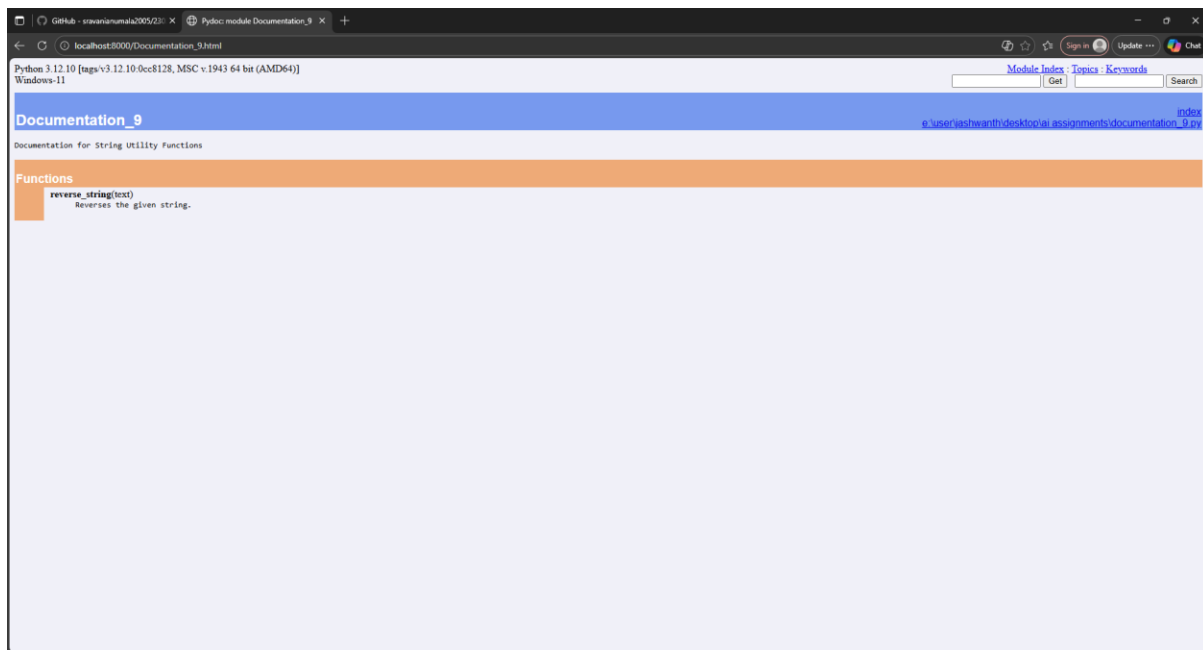
```
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:\User\Jashwanth\desktop\AI ASSIGNMENTS\Documentation_9.py"
PS E:\User\Jashwanth\desktop\AI ASSIGNMENTS> python -m pydoc Documentation_9
Help on module Documentation_9:

NAME
    Documentation_9

FUNCTIONS
    reverse_string(text)
        Reverses the given string.

        Args:
            text (str): Input string.

        Returns:
            str: Reversed version of the input string.
```

OBSERVATION:

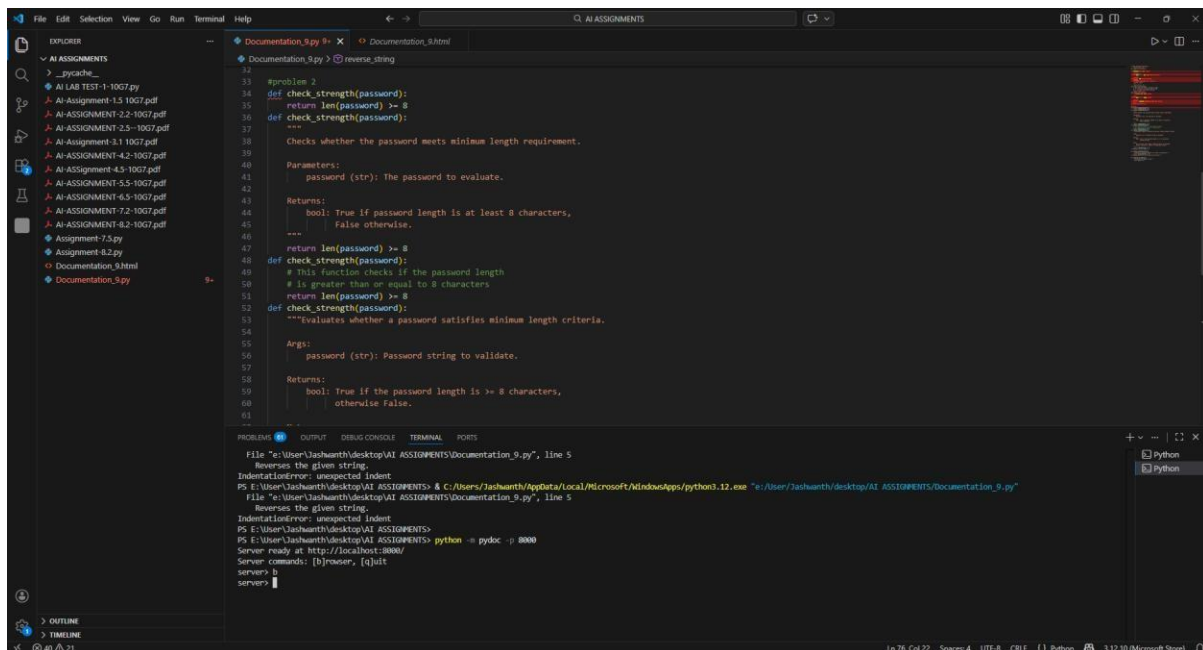
The function `reverse_string(text)` is written multiple times with proper documentation strings explaining parameters and return values. In the terminal, the `pydoc` command is executed, and it successfully displays the function documentation. This shows that the docstring is correctly written and Python is able to generate documentation from it.

PROBLEM-2:

PROMPT:

```
def check_strength(password):  
    return len(password) >= 8
```

CODE:



OBSERVATION:

The function `check_strength(password)` includes structured Google-style documentation that clearly explains its purpose, input parameter, and return value. It specifies that the function checks whether the password length is at least 8 characters. The documentation contains sections like **Args** (describing the password as a string input) and **Returns** (indicating a boolean value: True if the password meets the minimum length requirement, otherwise False). This format improves clarity, readability, and maintainability of the code.

PROBLEM-3:

PROMPT: reate a module `math_utils.py` with functions:

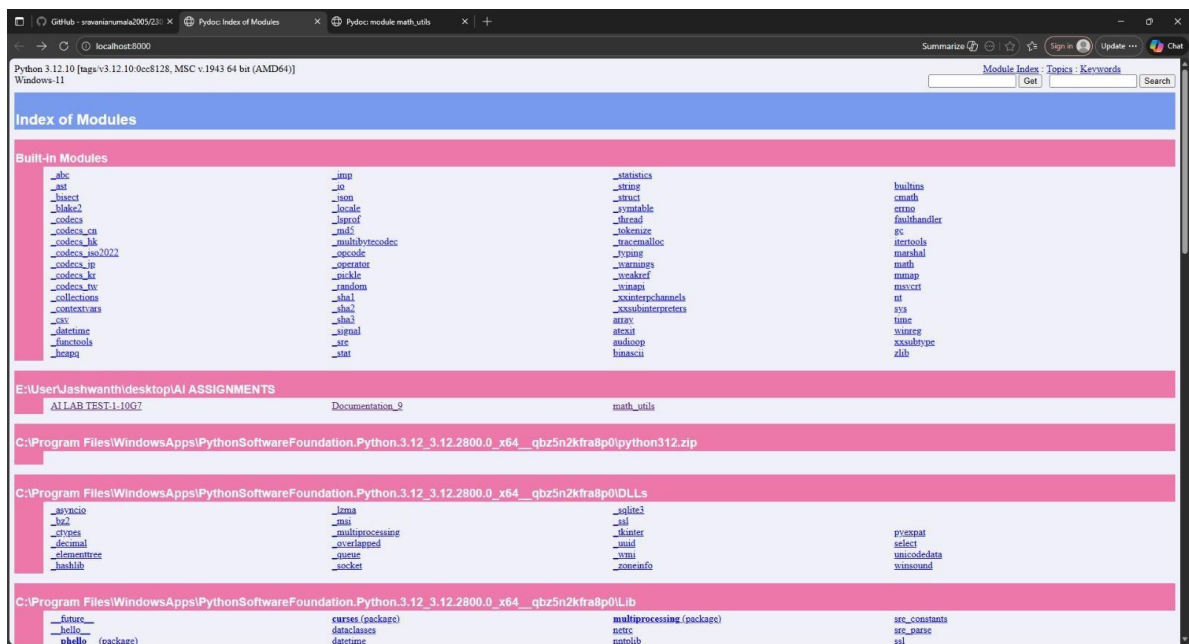
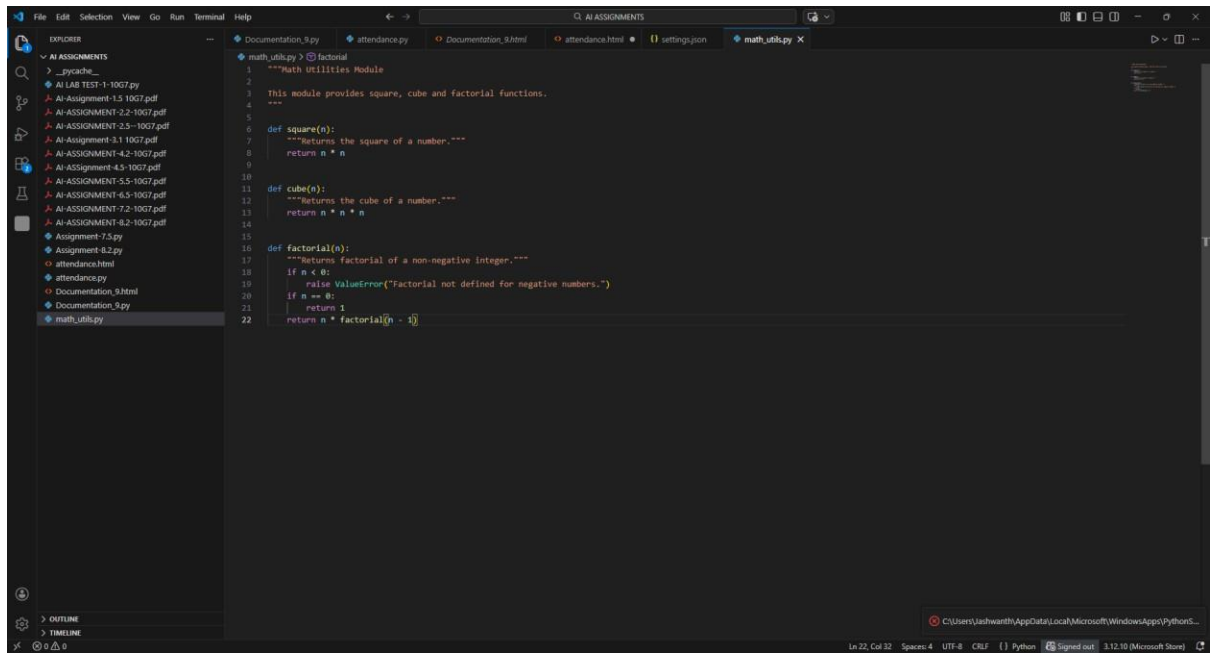
`square(n)`

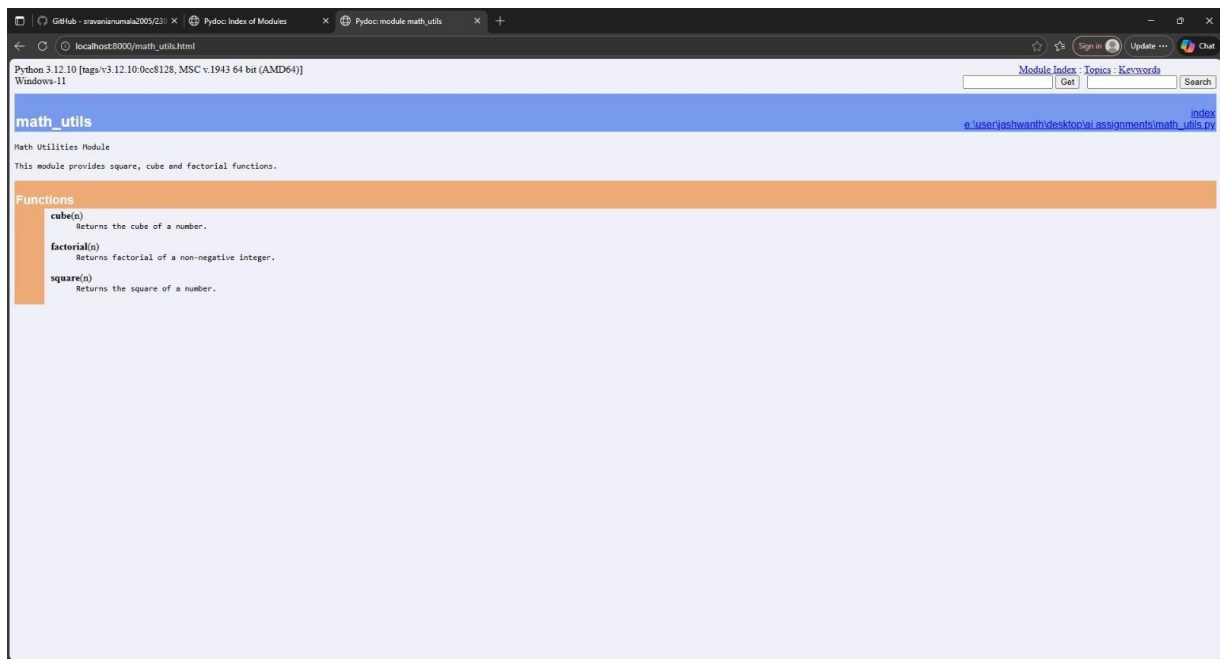
`cube(n)`

`factorial(n)`

Generate docstrings automatically using AI tools.

CODE:





OBSERVATION:

The Python file contains three functions: `square(n)`, `cube(n)`, and `factorial(n)`, each implemented with proper docstrings explaining their purpose.

Using `pydoc`, the documentation is automatically generated and displayed in the browser. The generated HTML page clearly lists the module name, a brief module description, and the available functions along with their explanations. The Index of Modules page also confirms that `math_utils` is correctly detected and included among the available modules.

This demonstrates that properly written docstrings enable automatic documentation generation, improving code readability and maintainability.

PROBLEM-4

PROMPT:

`mark_present(student)`

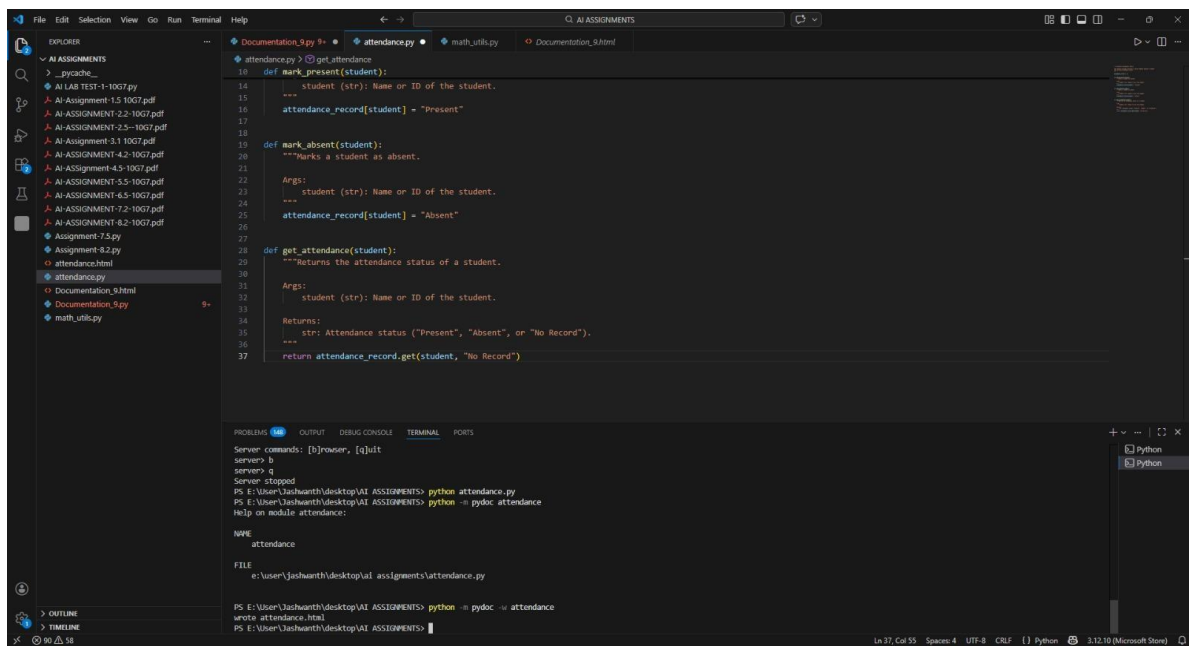
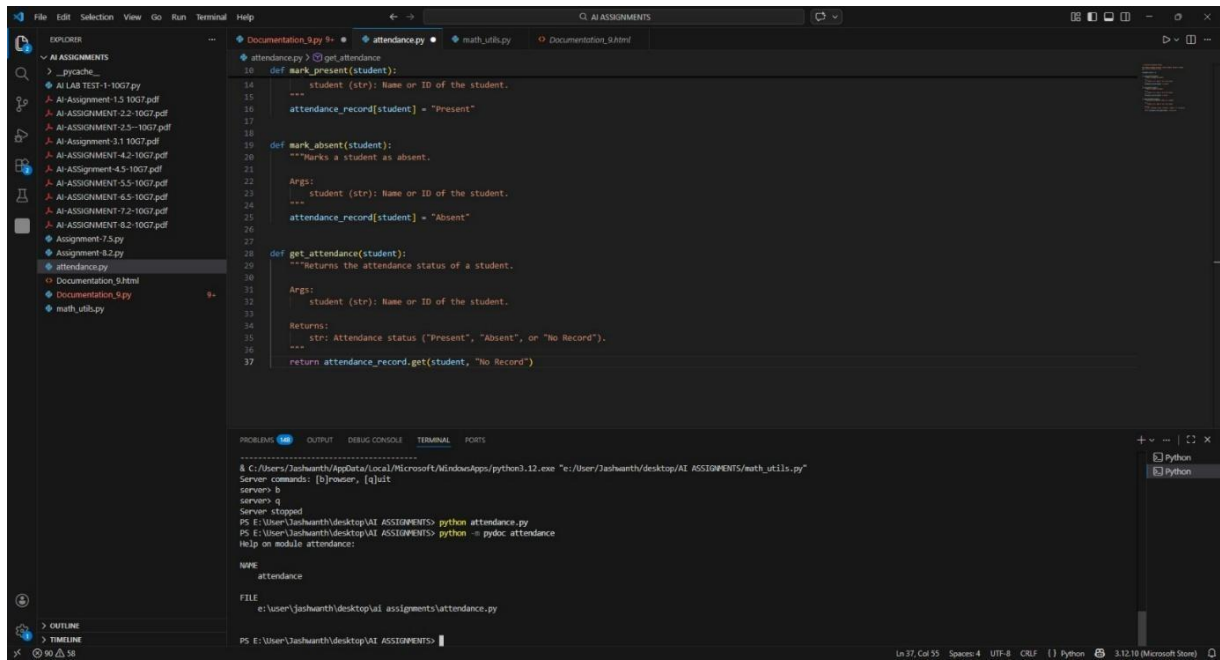
`mark_absent(student)`

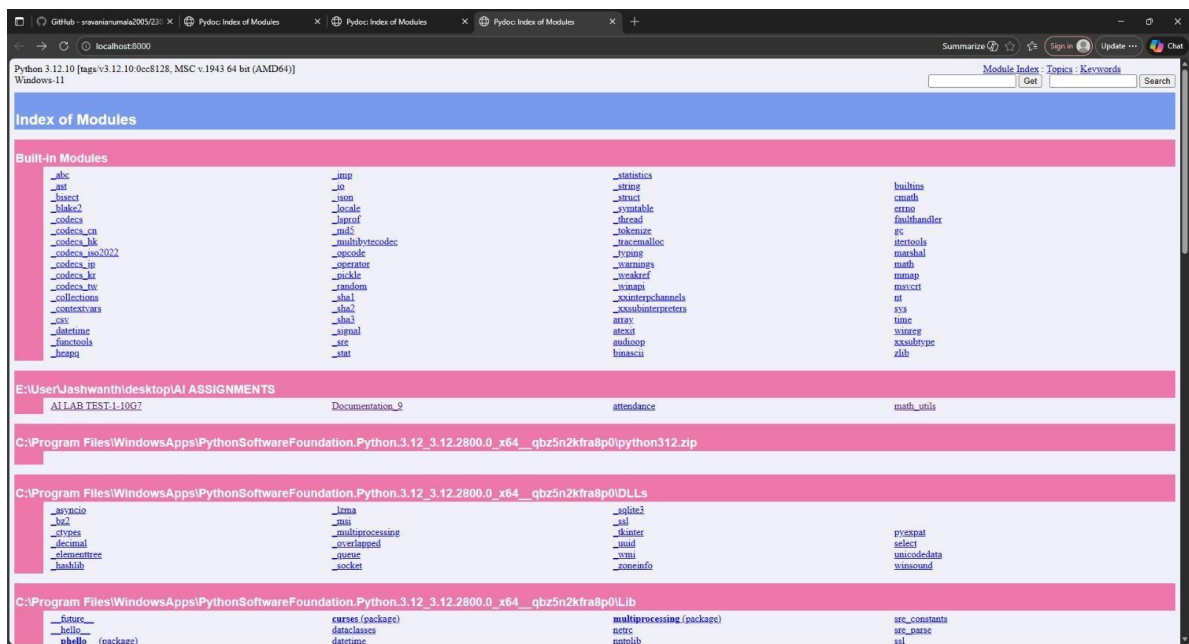
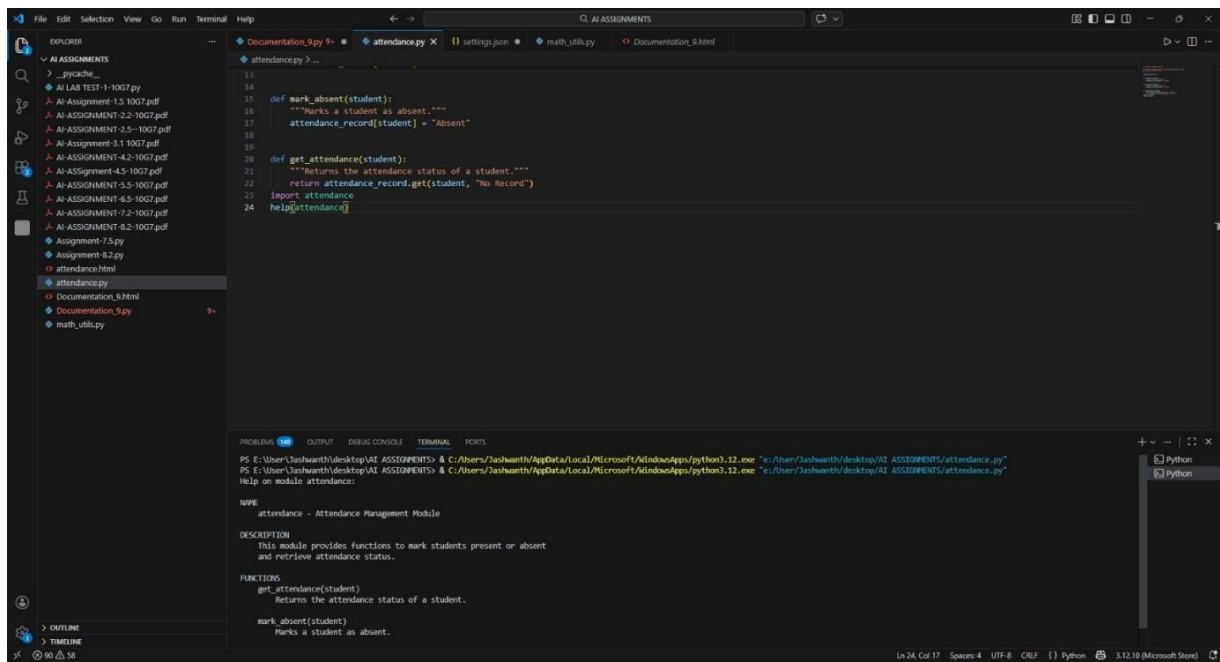
`get_attendance(student)`

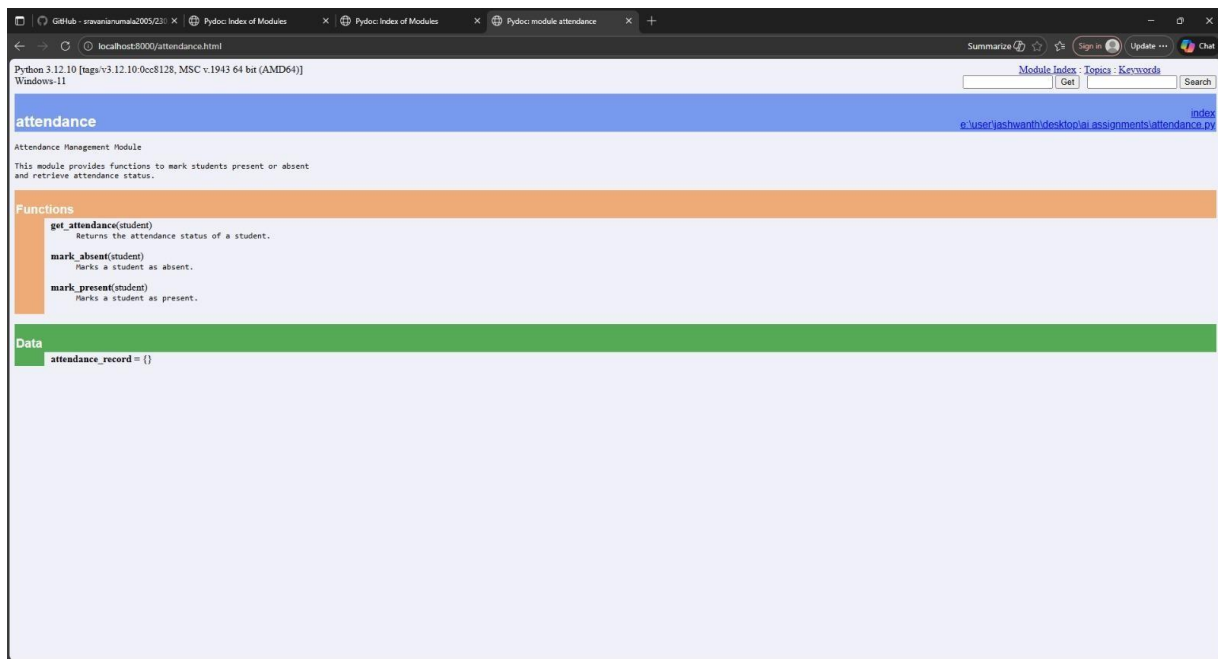
Add proper docstrings.

Generate and view documentation in terminal and browse

CODE:







OBSERVATION:

The module includes a clear title and a brief description stating that it manages student attendance. The documentation automatically lists the available functions such as `get_attendance(student)`, `mark_absent(student)`, and `mark_present(student)`, along with short explanations of their purposes.

Additionally, the global data variable `attendance_record = {}` is displayed under the Data section, indicating that pydoc also documents module-level variables.

This confirms that properly written docstrings and structured code enable automatic and organized documentation generation using pydoc, improving clarity and maintainability of the module.

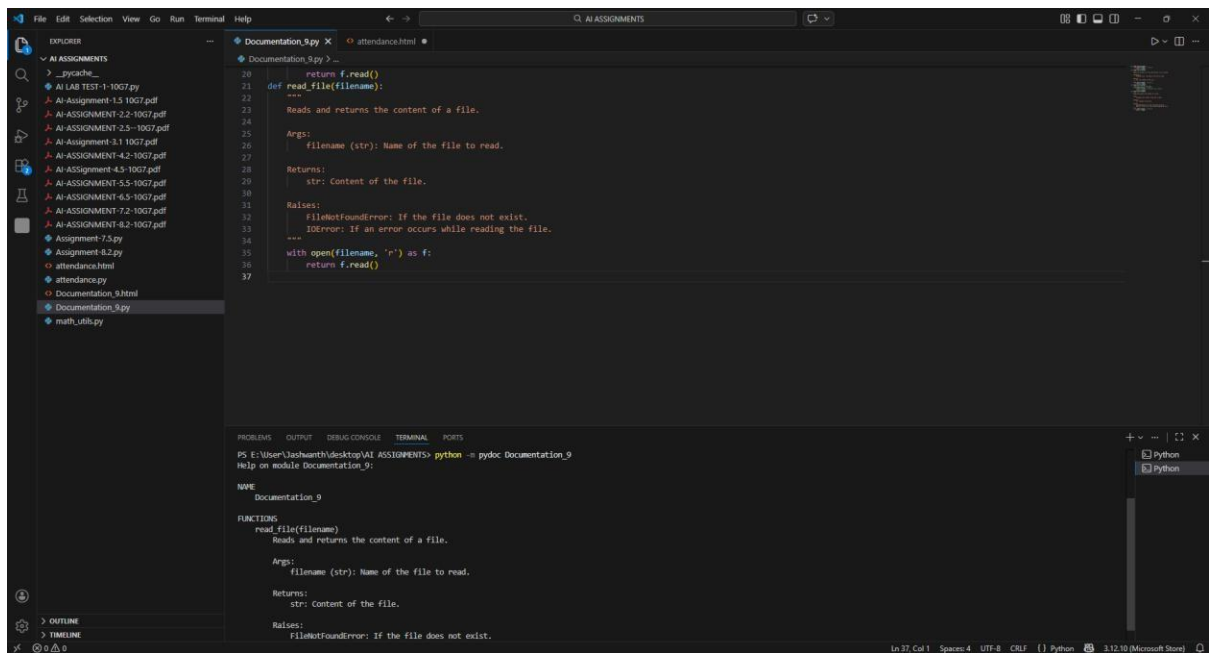
PROBLEM-5

PROMPT:

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()
```

Write documentation using all three formats.

CODE:



OBSERVATION:

The documentation was generated using pydoc, and the module details were displayed successfully in the terminal. This confirms that correctly written docstrings are recognized by Python.